

Deep Learning: Theory and Practice

Deep Learning - Practical Considerations

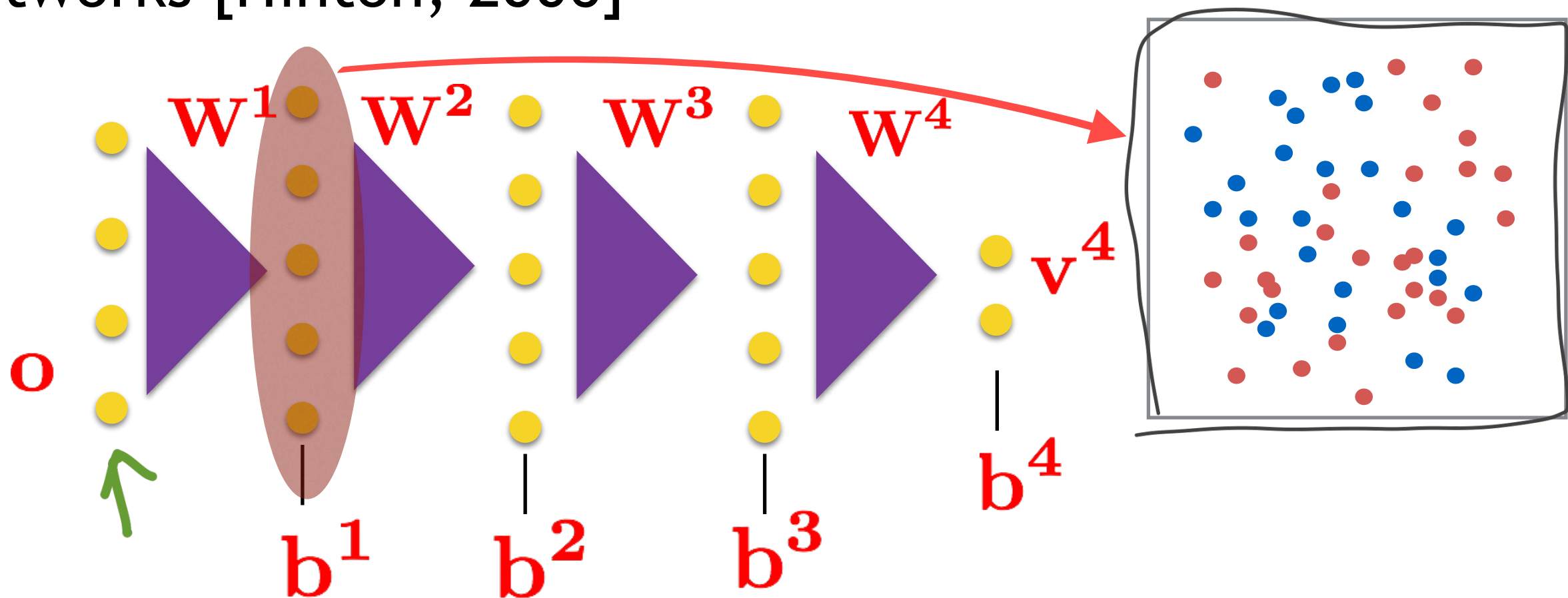
02-04-2020

deeplearning.cce2020@gmail.com



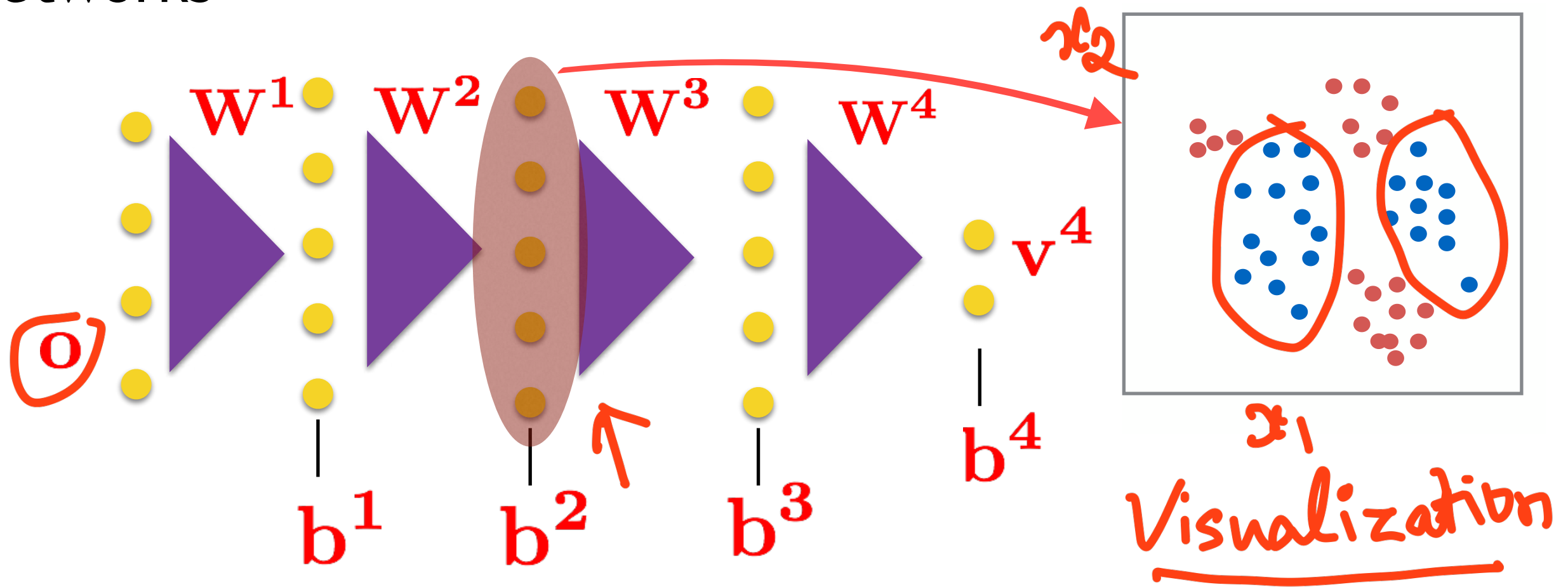
Deep Networks Intuition

Neural networks with multiple hidden layers - Deep networks [Hinton, 2006]



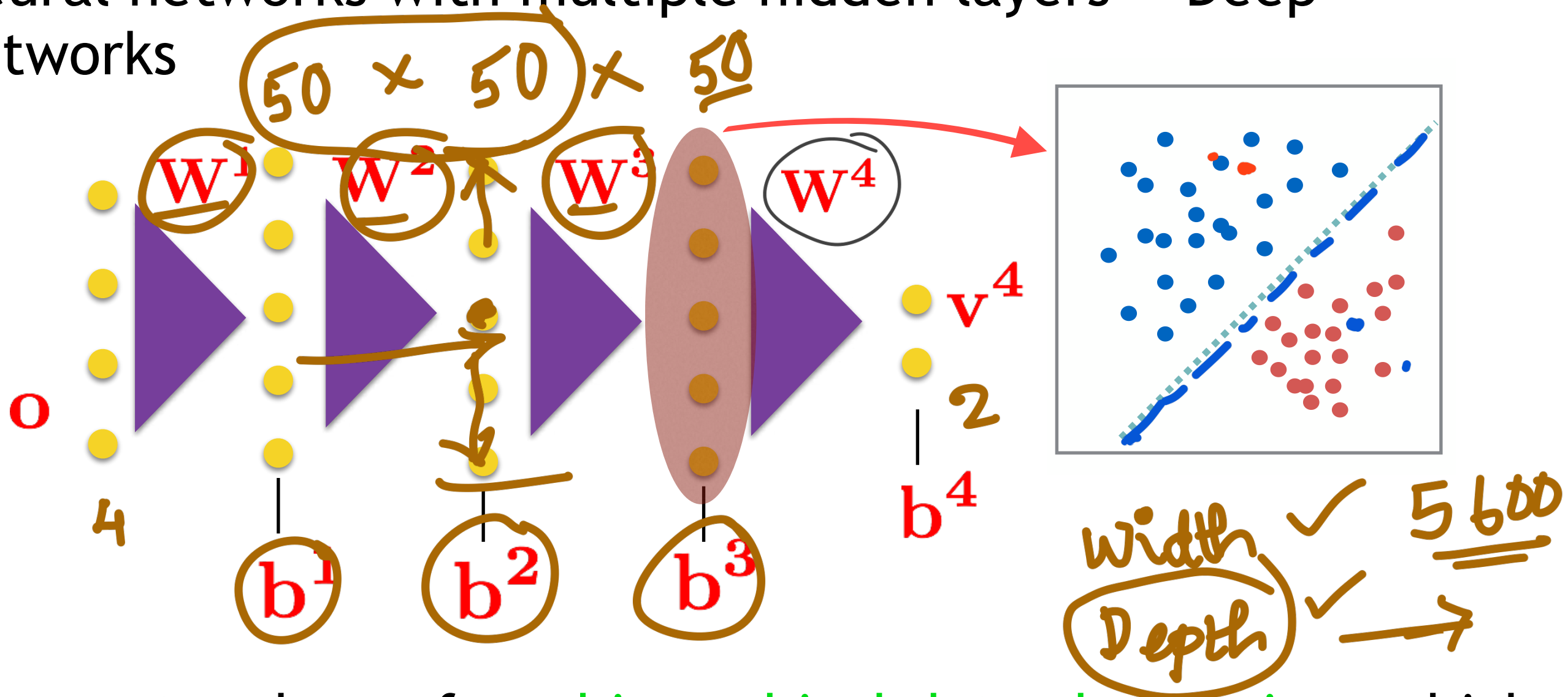
Deep Networks Intuition

Neural networks with multiple hidden layers - Deep networks



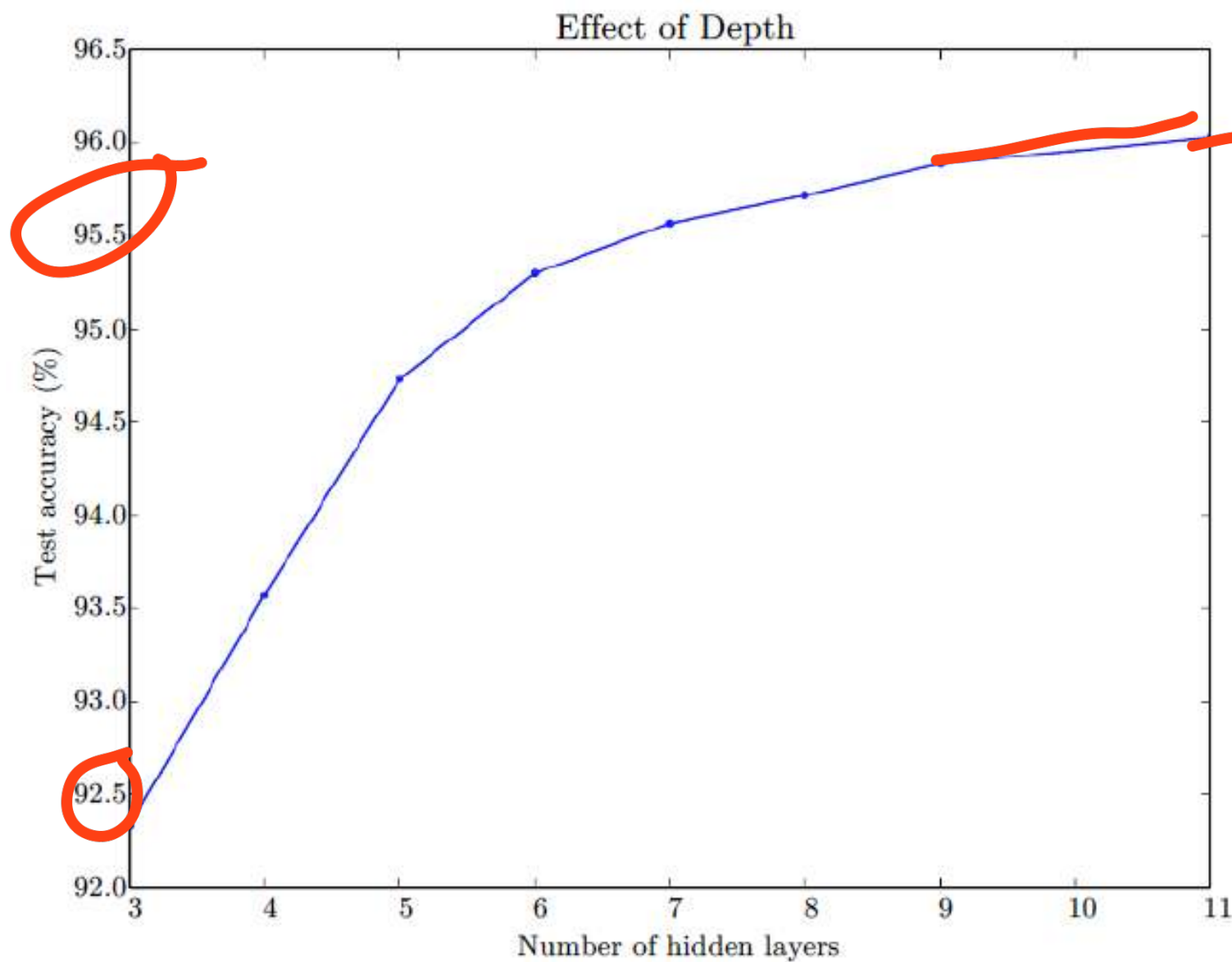
Deep Networks Intuition

Neural networks with multiple hidden layers - Deep networks



Deep networks perform hierarchical data abstractions which enable the non-linear separation of complex data samples.

Need for Depth

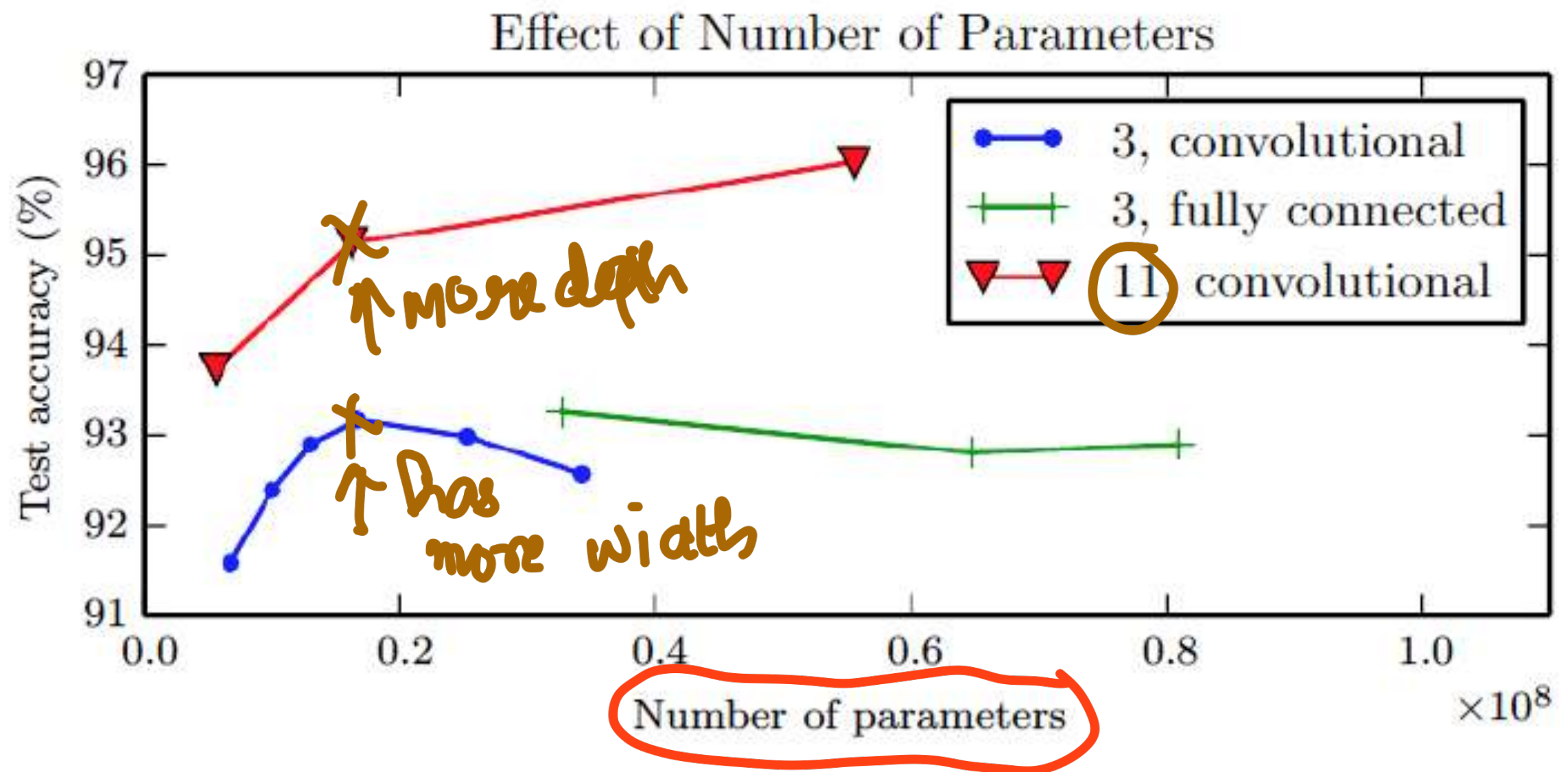


MNIST

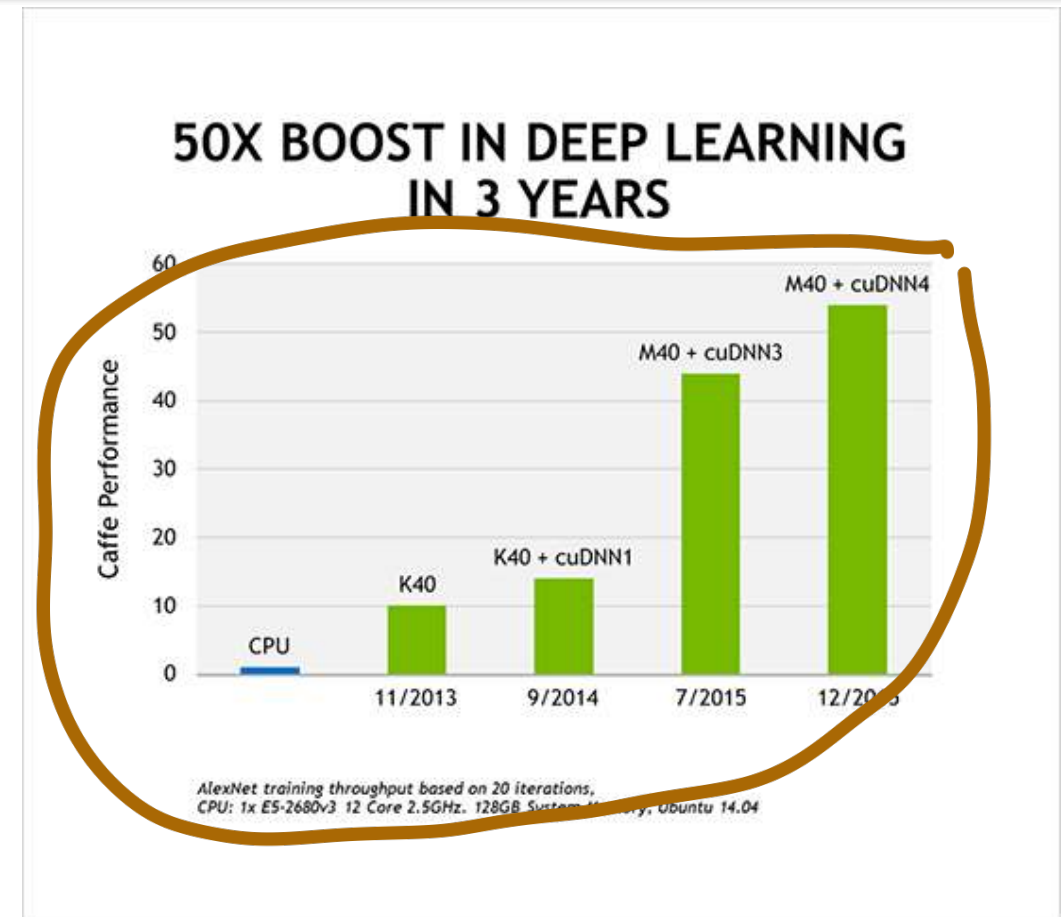
$$\mathbf{h}^{(1)} = g^{(1)} \left(\mathbf{W}^{(1)\top} \mathbf{x} + \mathbf{b}^{(1)} \right)$$

$$\mathbf{h}^{(2)} = g^{(2)} \left(\mathbf{W}^{(2)\top} \mathbf{h}^{(1)} + \mathbf{b}^{(2)} \right)$$

Need for Depth



Deep Networks



- Are these networks trainable ?
 - Advances in computation and processing
 - **Graphical processing units** (GPUs) performing multiple parallel multiply accumulate operations.
 - Large amounts of supervised data sets

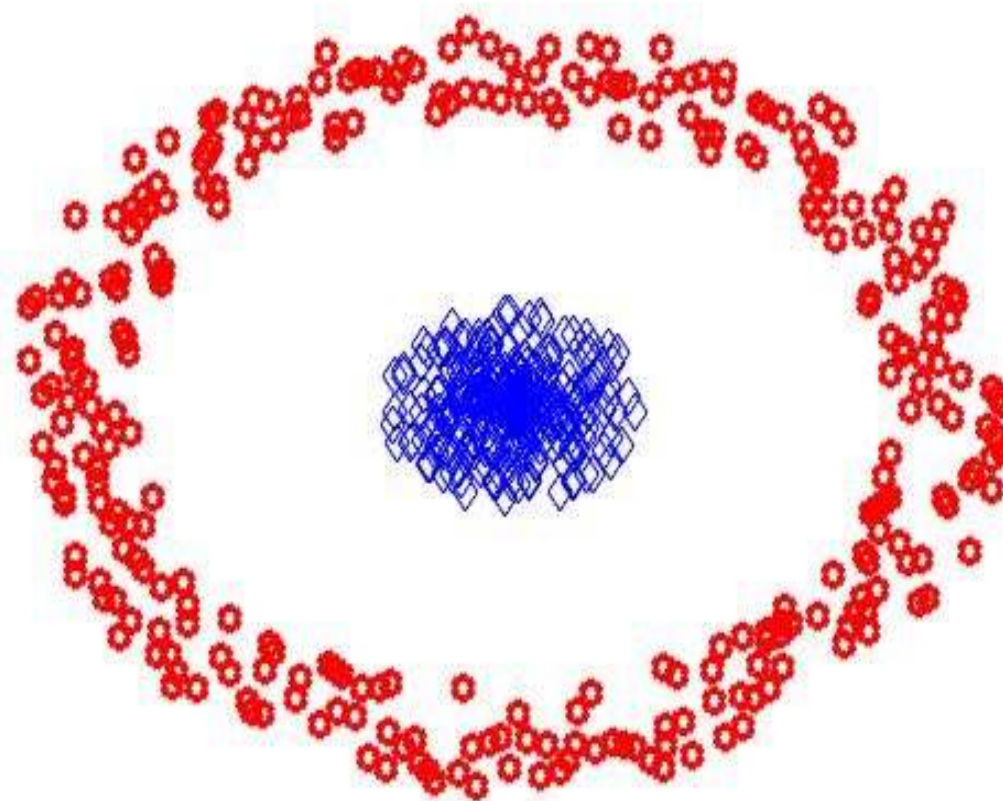
Deep Networks

- Will the networks generalize with deep networks *← Deviation from training*
 - DNNs are quite data hungry and performance improves by increasing the data.
 - Generalization problem is tackled by providing training data from all possible conditions.
 - Many artificial data augmentation methods have been successfully deployed
 - Providing the state-of-art performance in several real world applications.

Representation Learning in Deep Networks

- The input data representation is one of most important components of any machine learning system. $\|y - t\|^2$

Cartesian Coordinates



Non-linear

Polar Coordinates

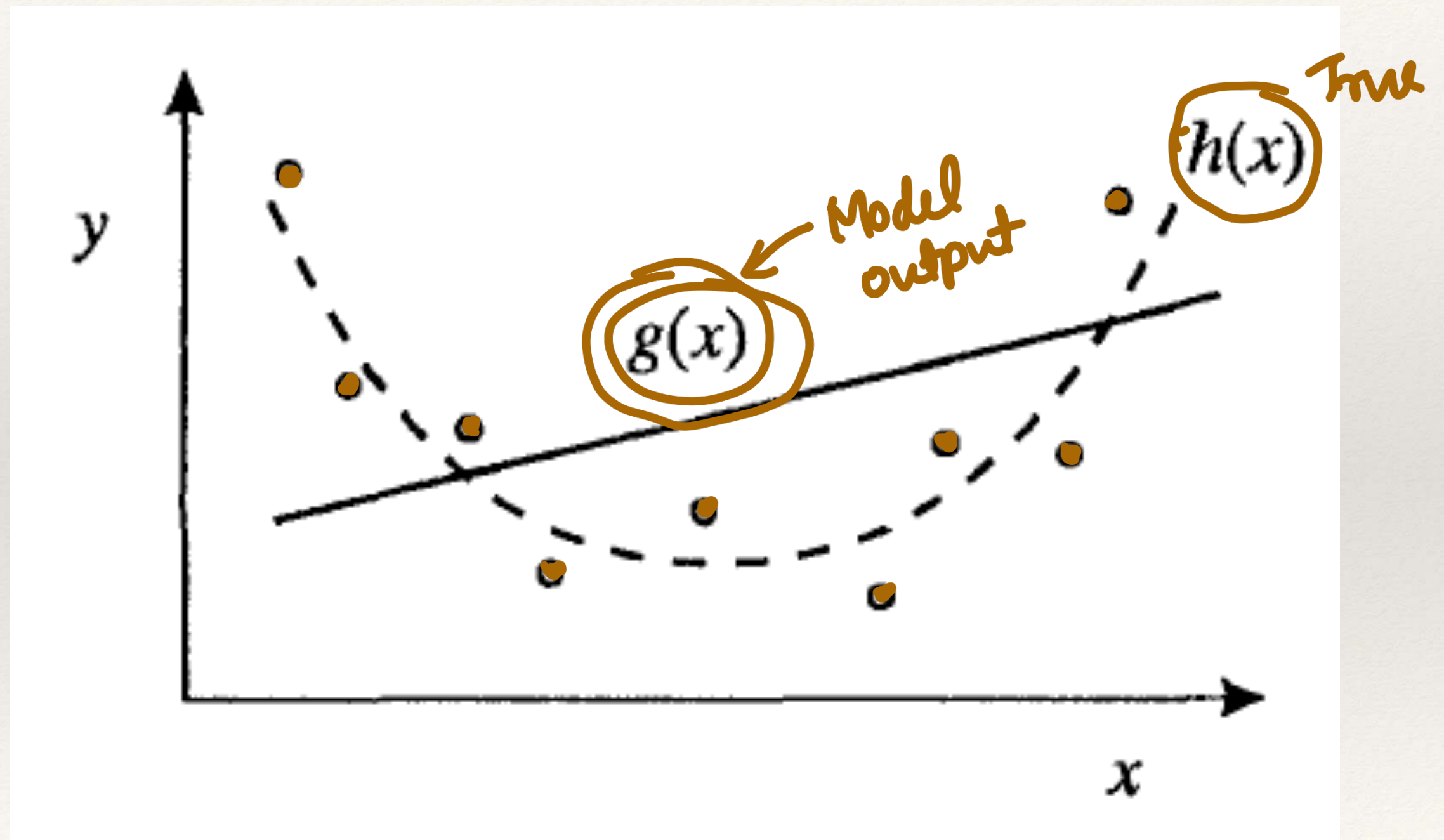


$\|\log y - \log t\|^2$

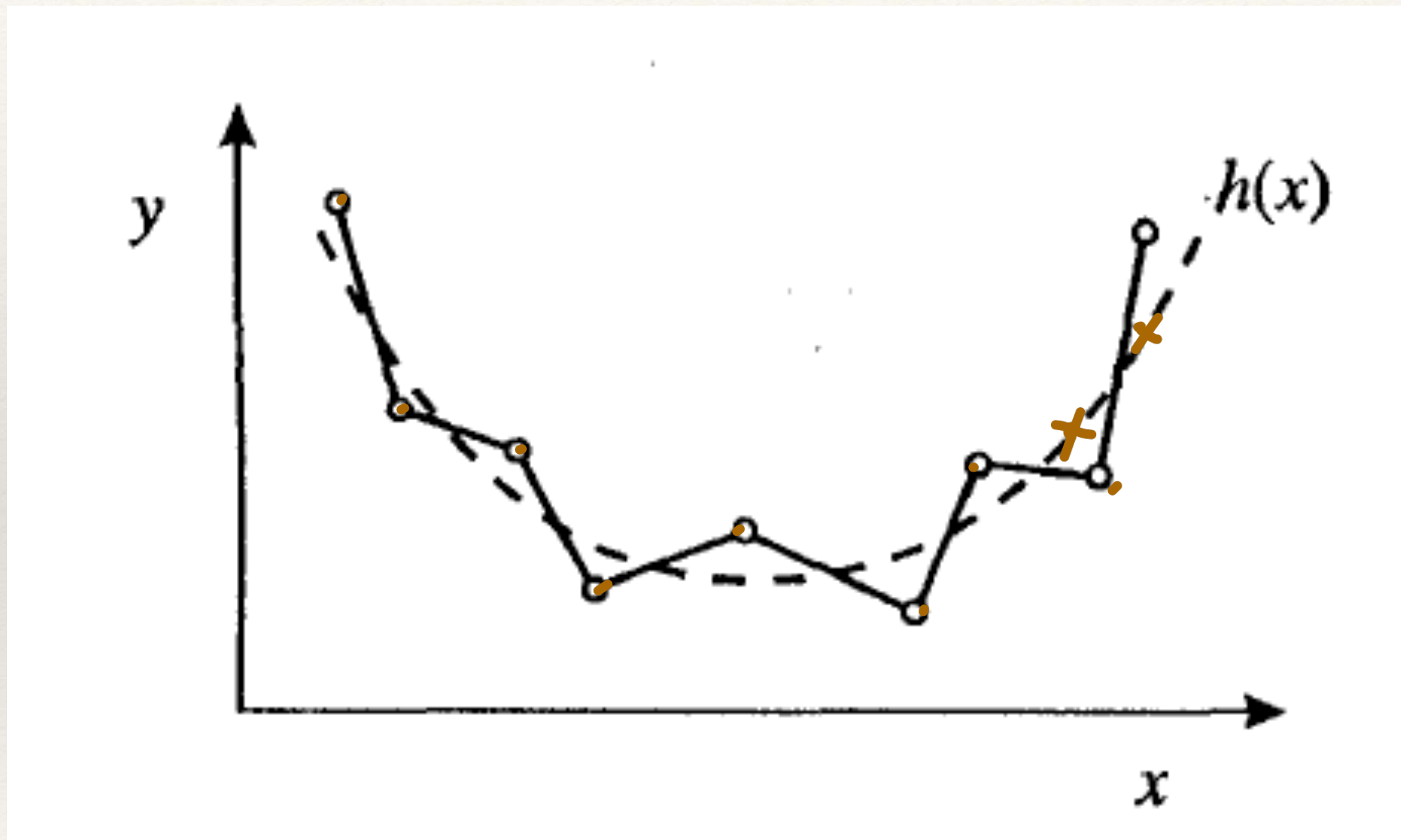
Representation Learning in Deep Networks

- The input data representation is one of most important components of any machine learning system.
 - Extract factors that enable classification while suppressing factors which are susceptible to noise.
- Finding the right representation for real world applications - substantially challenging.
 - ✱ Deep learning solution - build complex representations from simpler representations.
 - ✱ The dependencies between these hierarchical representations are refined by the target.

Underfit



Overfit → Lack of Generalization.
↑



Avoiding OverFitting In Practice



Regularization

$E_D(W)$ ← data dependent error function.

$$\|W'\|_2^2 = \sum_i \sum_j (w'_{ij})^2$$
$$\|W''\|_2^2 = \sum_p \sum_q (w''_{pq})^2$$

Overfitting ← following training data too closely.

Learn patterns

$E_W(W)$ ← Data independent loss function.
Model dependent

Backpropagation

$E(W)$

$E_D(W)$

λ

$E_W(W)$

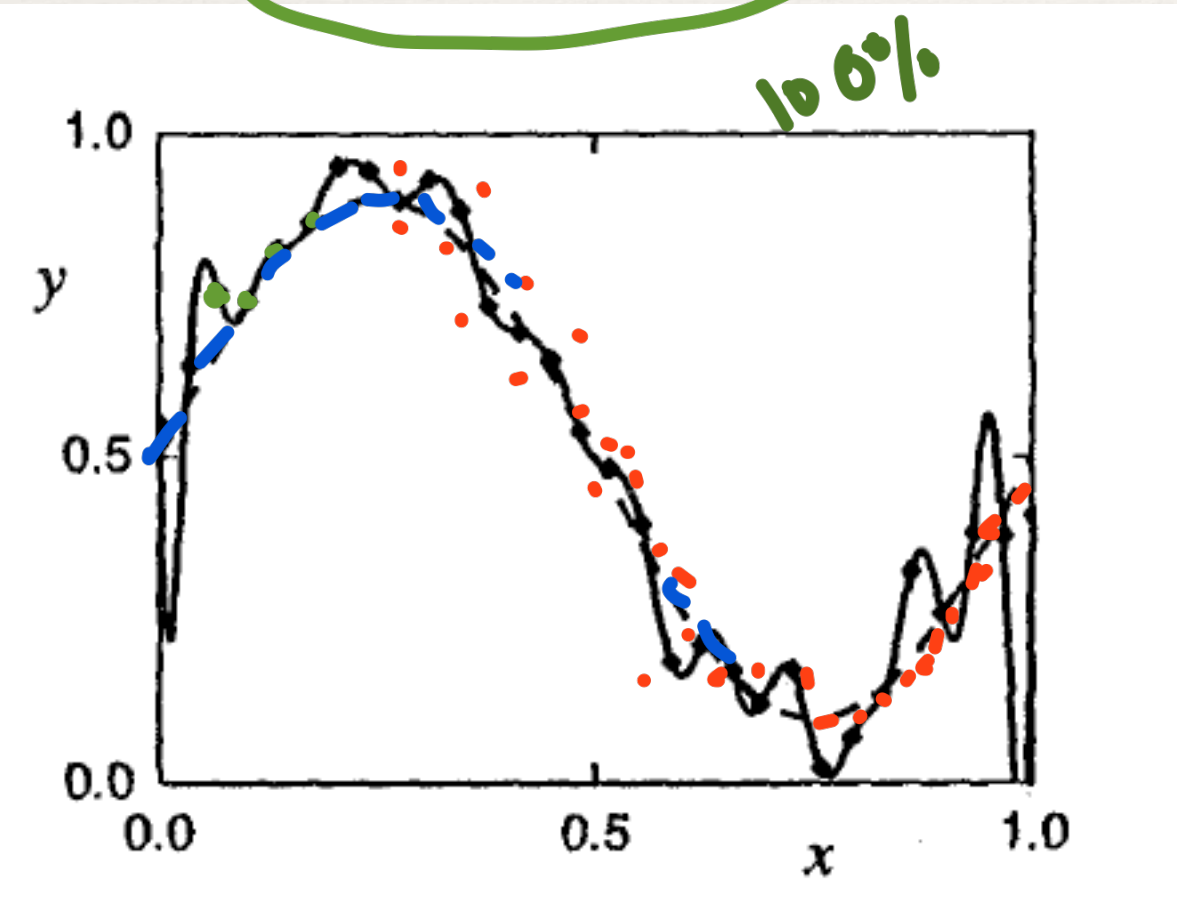
Regularization Parameter.

λ - small
Overfitting
 λ - large (underfit)

Weight Decay Regularization $E_W(W) = \|W\|_2^2$

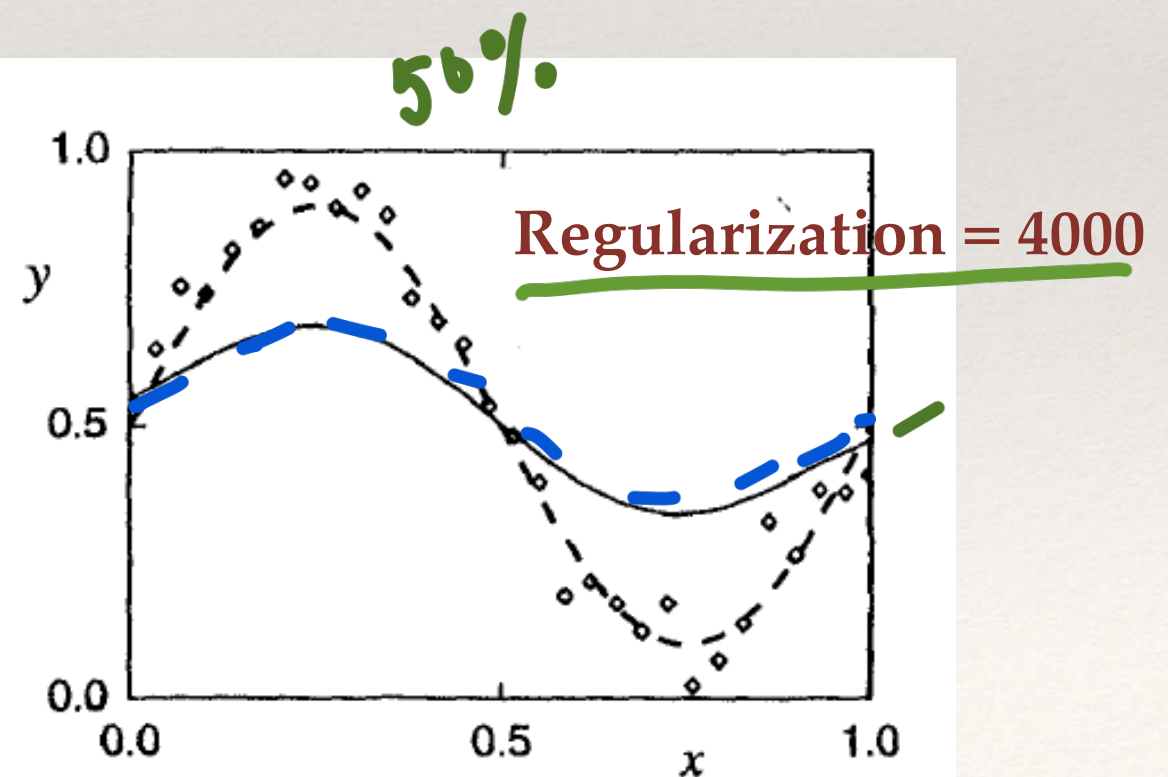
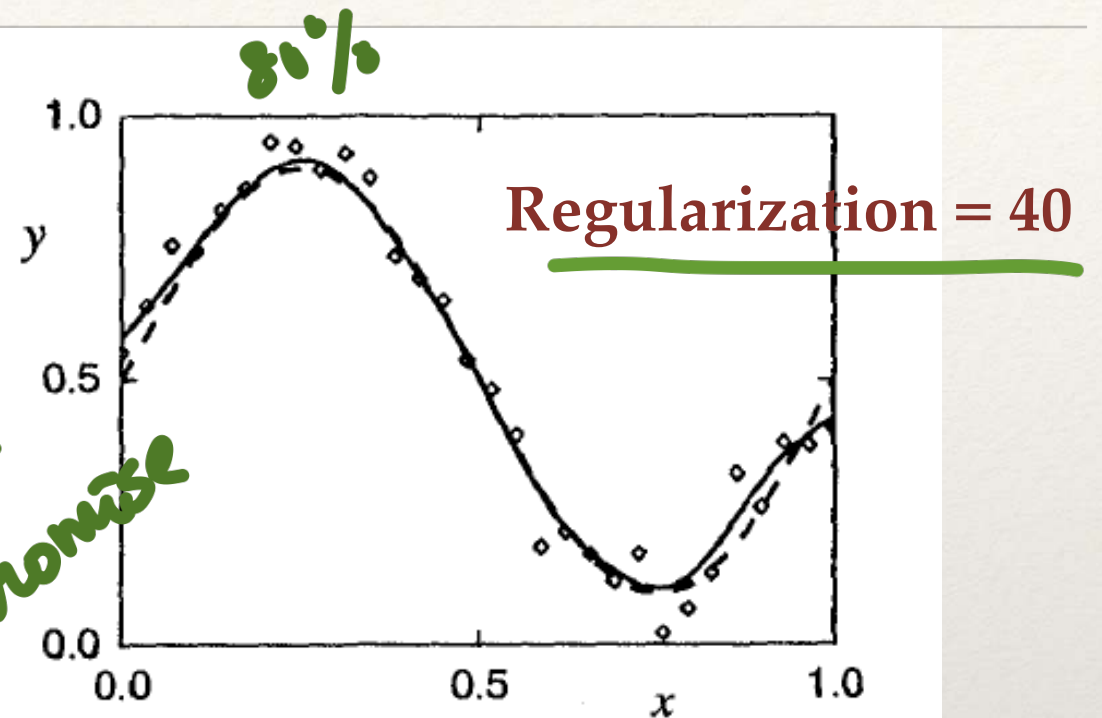
I. Weight Decay Regularization

Regularization = 0



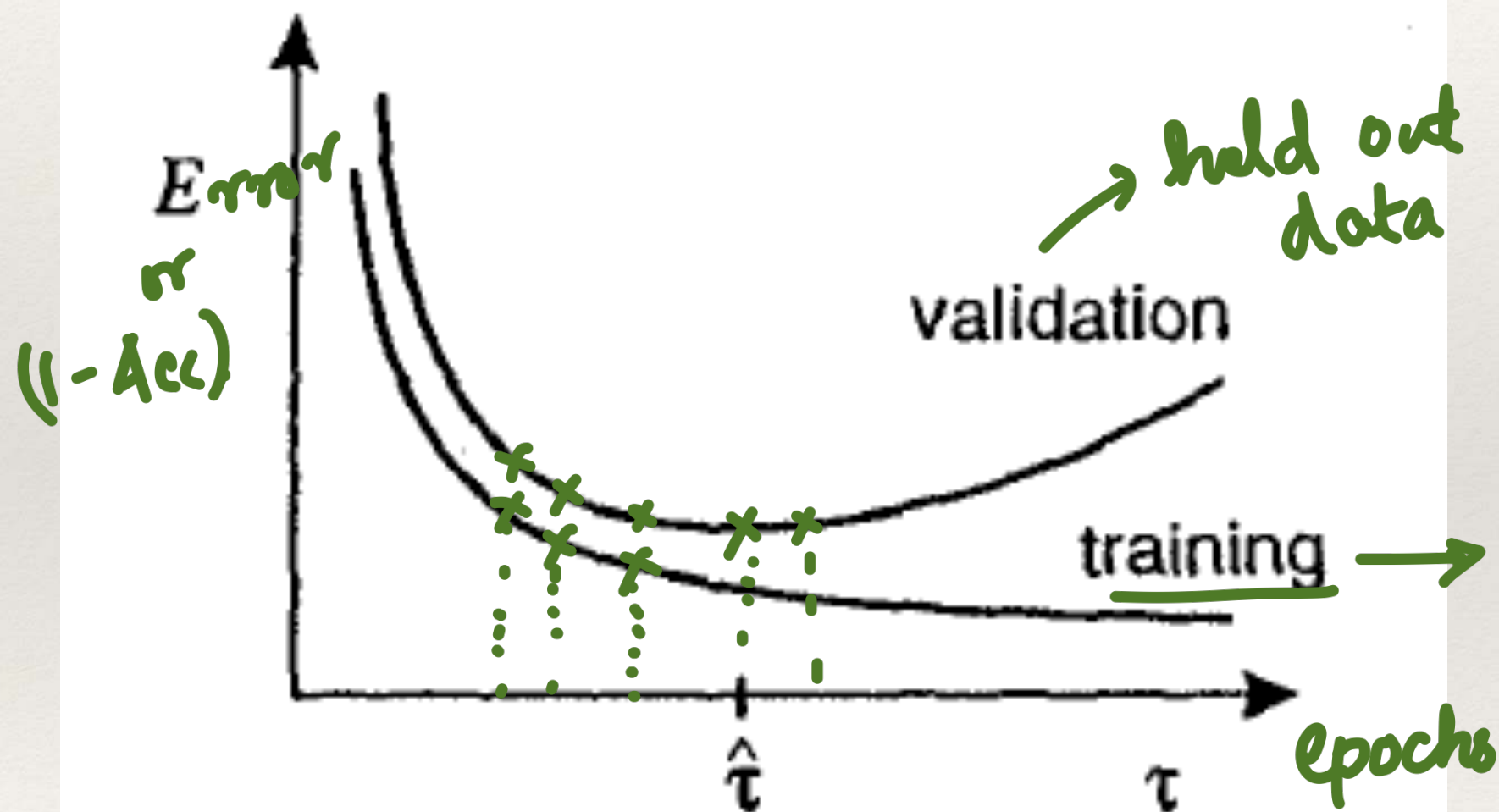
λ - small number gives you best compromise.

Right compromise



II.

Early Stopping



Most Popular in Practice

Stop training
when validation
performance
does not improve

III. Batch Normalization

Batch Normalization: Accelerating Deep Network Training by
Reducing Internal Covariate Shift

2015

Sergey Ioffe

Google Inc., sioffe@google.com

Christian Szegedy

Google Inc., szegedy@google.com

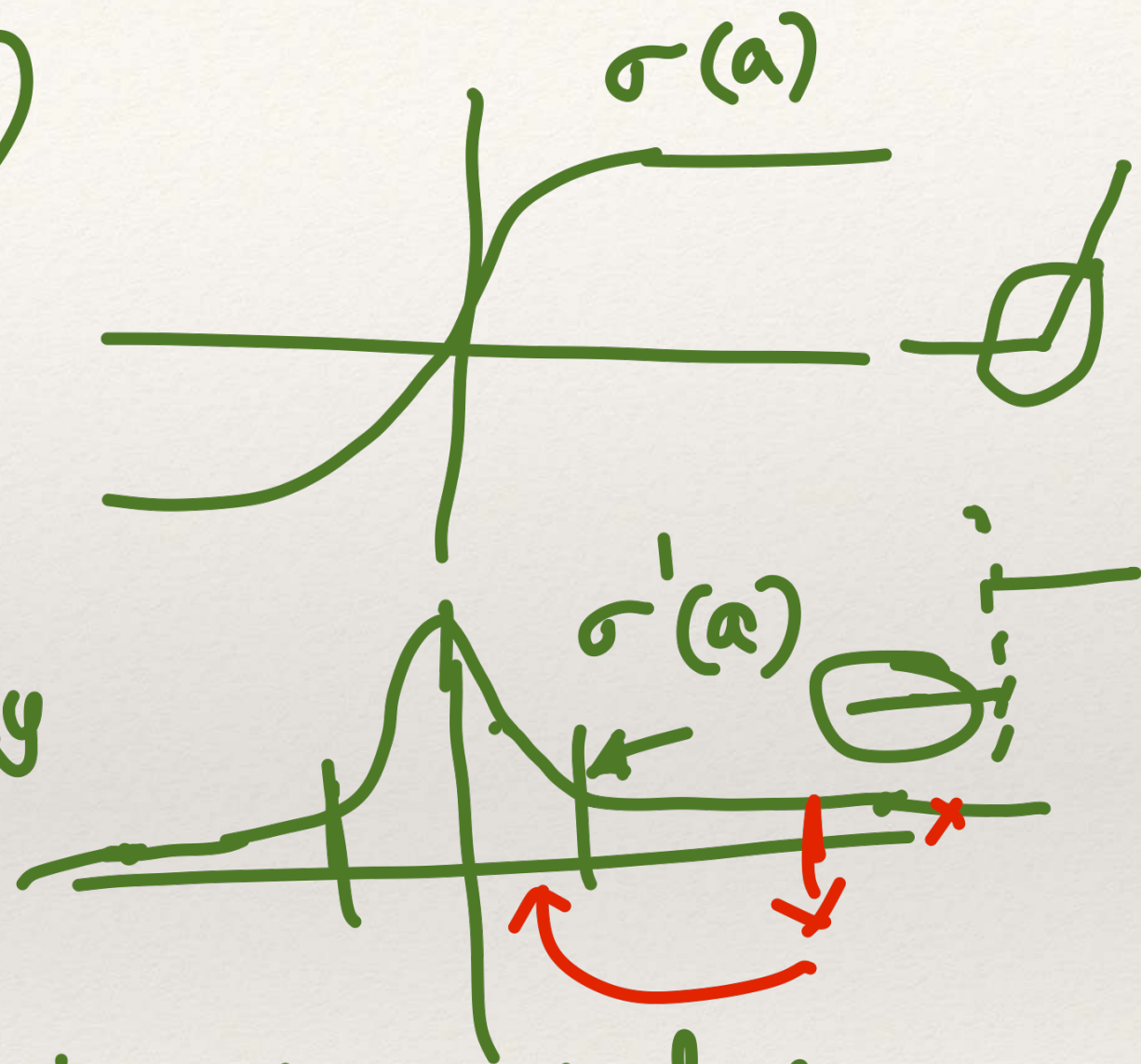
Sigmoid as non-linearity (Two layers)

$$h^2 = (W^2 \underbrace{\sigma(W^1 \underline{x} + b^1)}_{z^1} + b^2)$$

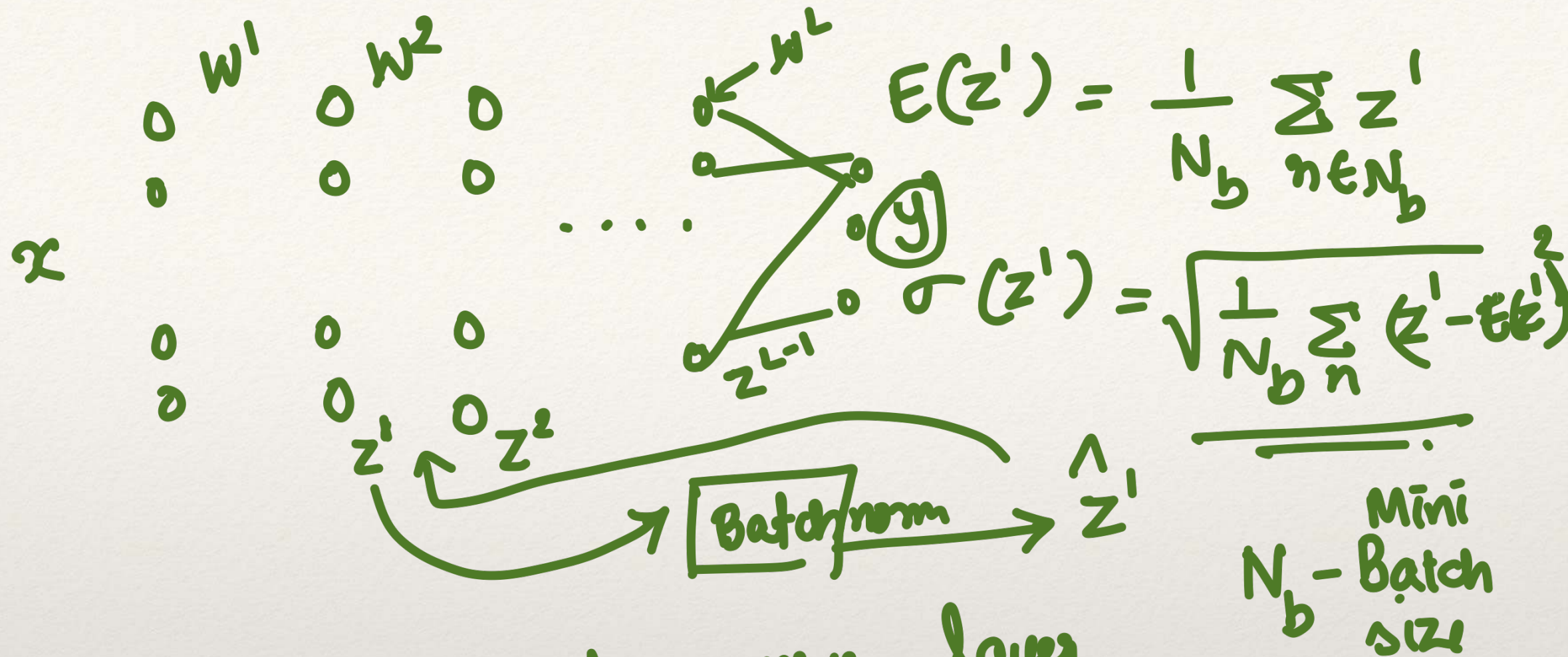
Model saturation if weight values or inputs are very high.

Batch normalization at every layer.

$$\hat{\underline{z}}^1 = \frac{z^1 - E(z^1)}{\sigma(z^1)}$$



← This makes each layer output normalized.
(close to 0, have small variance only).



Can be done at every layer.

* higher learning rate

* Reduced number of epochs

should be $O(10^2)$

Effect of Batch Normalization

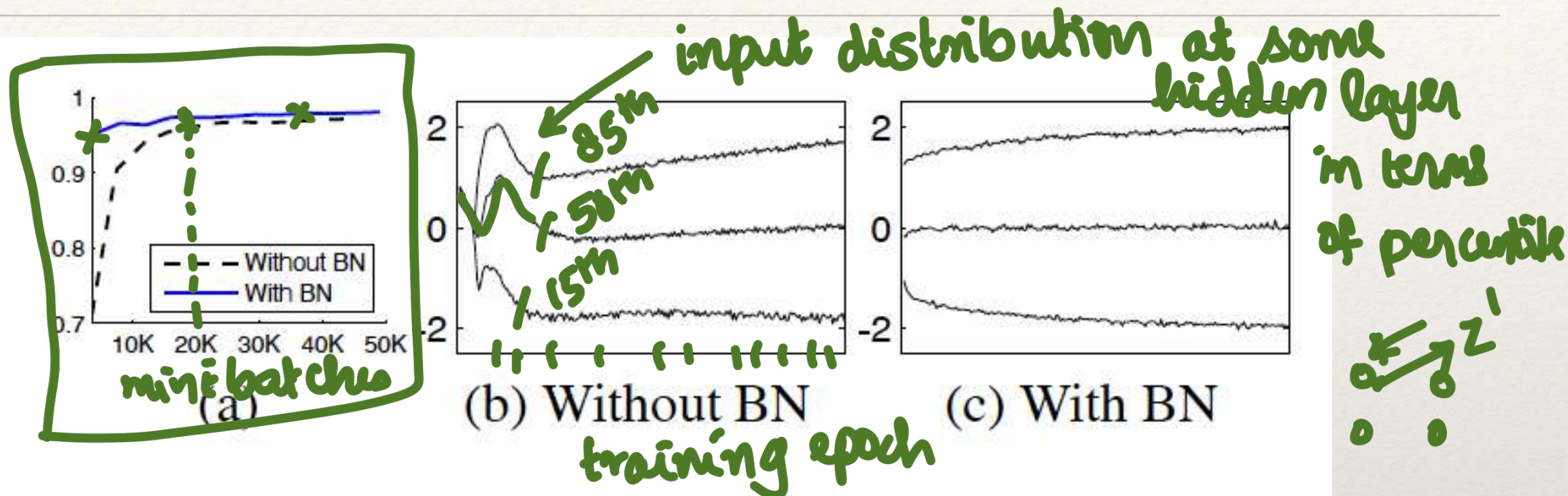


Figure 1: (a) The test accuracy of the MNIST network trained with and without Batch Normalization, vs. the number of training steps. Batch Normalization helps the network train faster and achieve higher accuracy. (b, c) The evolution of input distributions to a typical sigmoid, over the course of training, shown as $\{15, 50, 85\}$ th percentiles. Batch Normalization makes the distribution more stable and reduces the internal covariate shift.

$$\{z'\}_{n=1}^N$$

IV. Dropout Strategy in Neural Network Training

Dropout: A Simple Way to Prevent Neural Networks from Overfitting

Nitish Srivastava

Geoffrey Hinton

Alex Krizhevsky

Ilya Sutskever

Ruslan Salakhutdinov

Department of Computer Science

University of Toronto

10 Kings College Road, Rm 3302

Toronto, Ontario, M5S 3G4, Canada.

NITISH@CS.TORONTO.EDU

HINTON@CS.TORONTO.EDU

KRIZ@CS.TORONTO.EDU

ILYA@CS.TORONTO.EDU

RSALAKHU@CS.TORONTO.EDU

2013

Editor: Yoshua Bengio

Dropouts in Neural Networks

- The diagram illustrates the concept of dropout in neural networks. It consists of two parts: (a) Standard Neural Net and (b) After applying dropout.

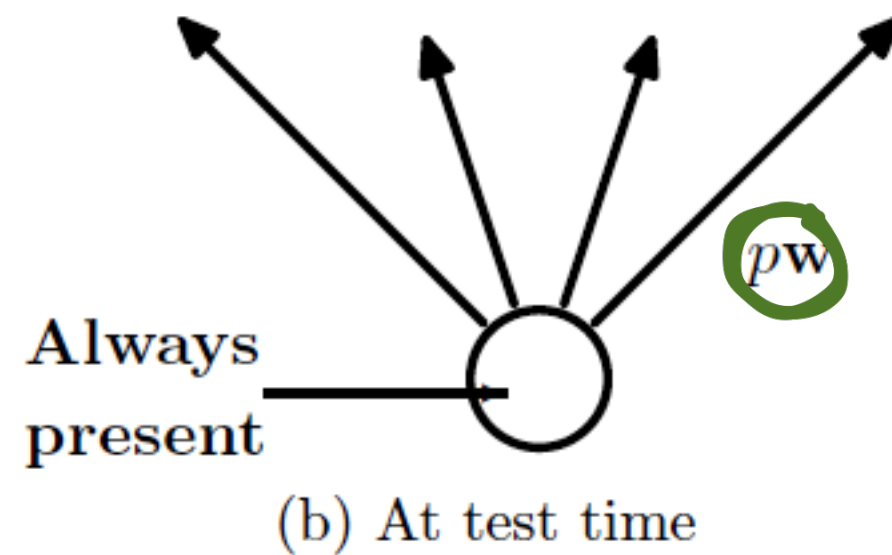
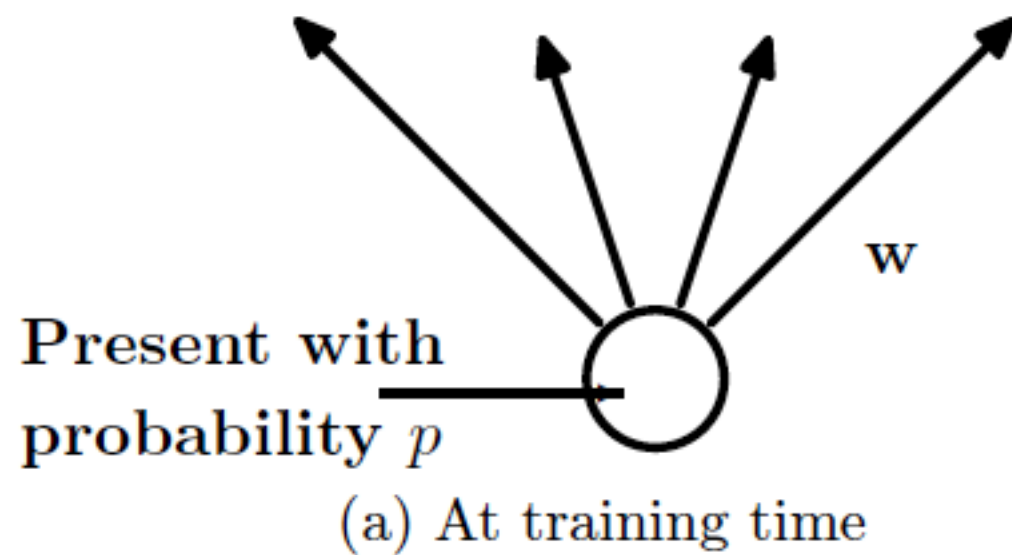
(a) Standard Neural Net: A fully connected neural network with three layers. The input layer has 4 nodes, the hidden layer has 4 nodes, and the output layer has 1 node. All nodes are connected to each other in a feed-forward manner. Handwritten notes in green ink state: "* Nodes (neurons) are too high to perform task" and "* Any random subset of neurons should still be able to predict o/p". A green arrow points to the connections between the input and hidden layers, labeled "feed forward connection".

(b) After applying dropout: The same neural network structure, but with some nodes and connections removed (indicated by red 'X' marks). The input layer has 2 active nodes, the hidden layer has 2 active nodes, and the output layer has 1 active node. Handwritten notes in red ink include: "probability of switching it off", "softmax", and "70x70". Green arrows point to the weights of the connections between the input and hidden layers, labeled $W^1_{100 \times 100}$ and $W^2_{100 \times 100}$. The output layer is labeled $W^2 = []$ and $W^1 = []$.

Fixed Dropout probability

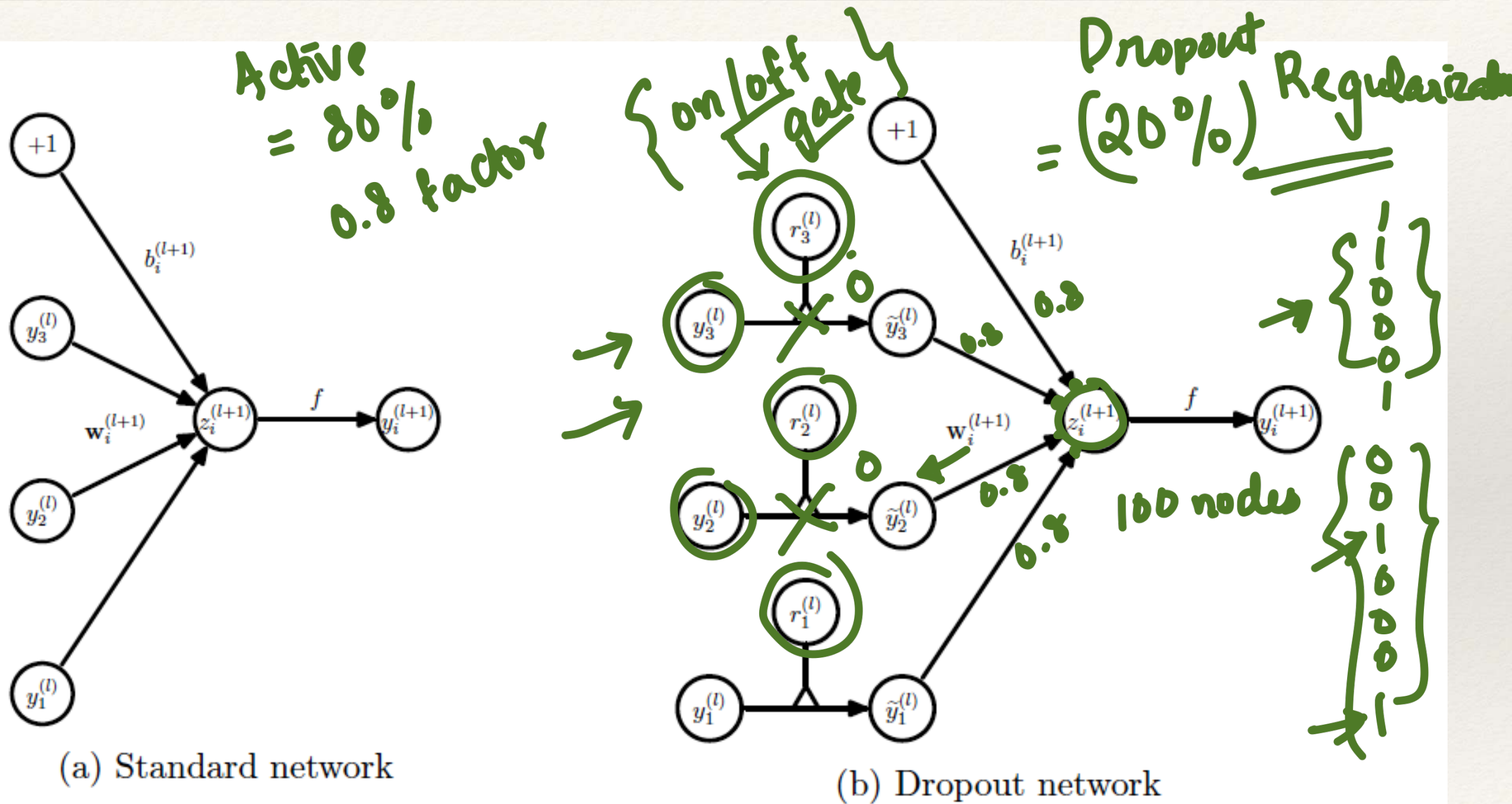
(Sparsity)

Dropout in Training and Test



Consistent o/p
in testing

Dropout Application



(a) Standard network

(b) Dropout network

Figure 3: Comparison of the basic operations of a standard and dropout network.

Effect of Dropouts

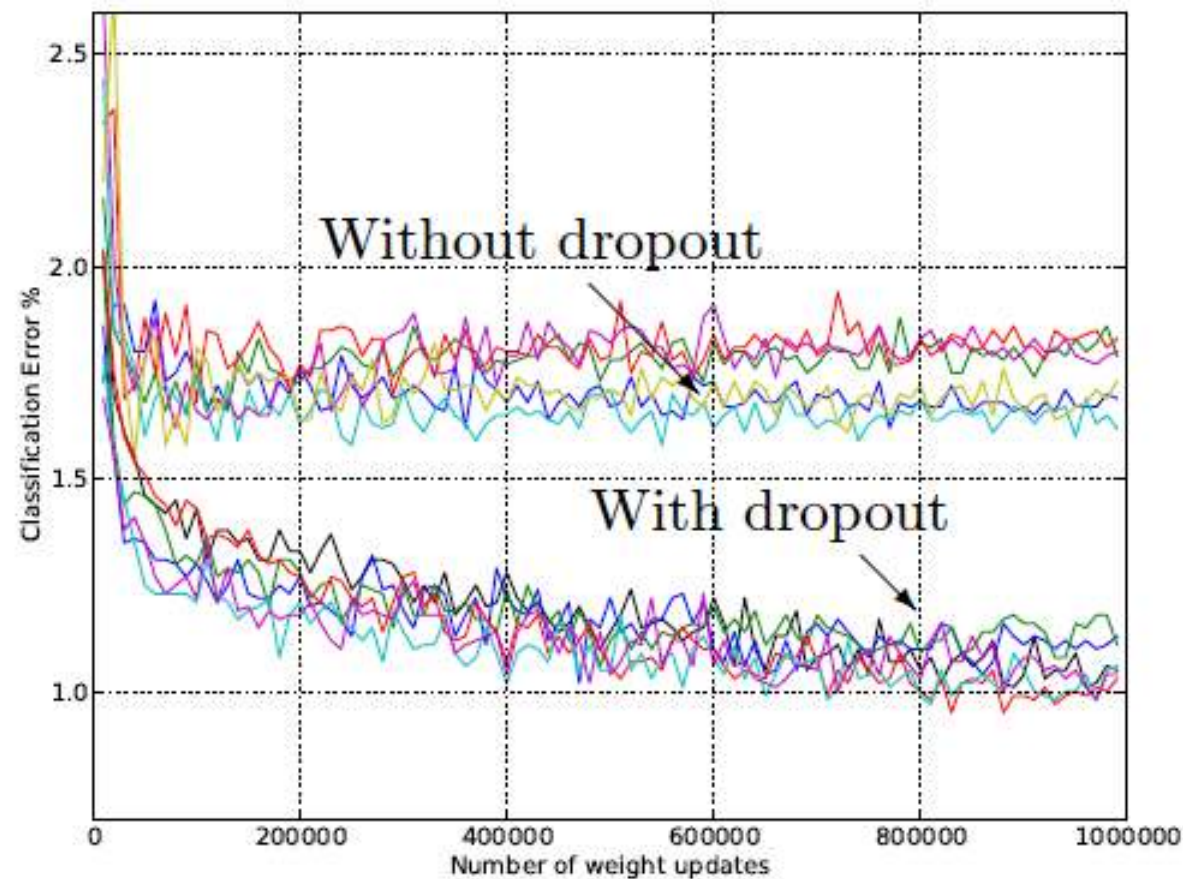
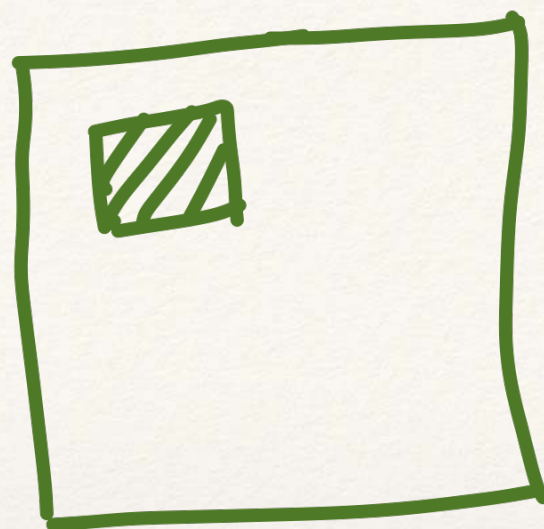


Figure 4: Test error for different architectures with and without dropout. The networks have 2 to 4 hidden layers each with 1024 to 2048 units.

Convolutional Neural Networks

CNNs



2-D patterns

$\sqrt{3}$

↓ Can I also have weights which are 2-D.
and local. [process parts of Image].

Convolution

Operations

$$\underline{\underline{Y^{(p)}(i, j)}} =$$

$$\sum_{m=0}^{M-1} \sum_{n=0}^{N-1} X(i+m, j+n) K^p(m, n)$$

$$K \in \mathbb{R}^{M \times N}$$

Learnable parameter

$$K^p(m, n)$$

$p=1 \dots P$

DNN



Weights



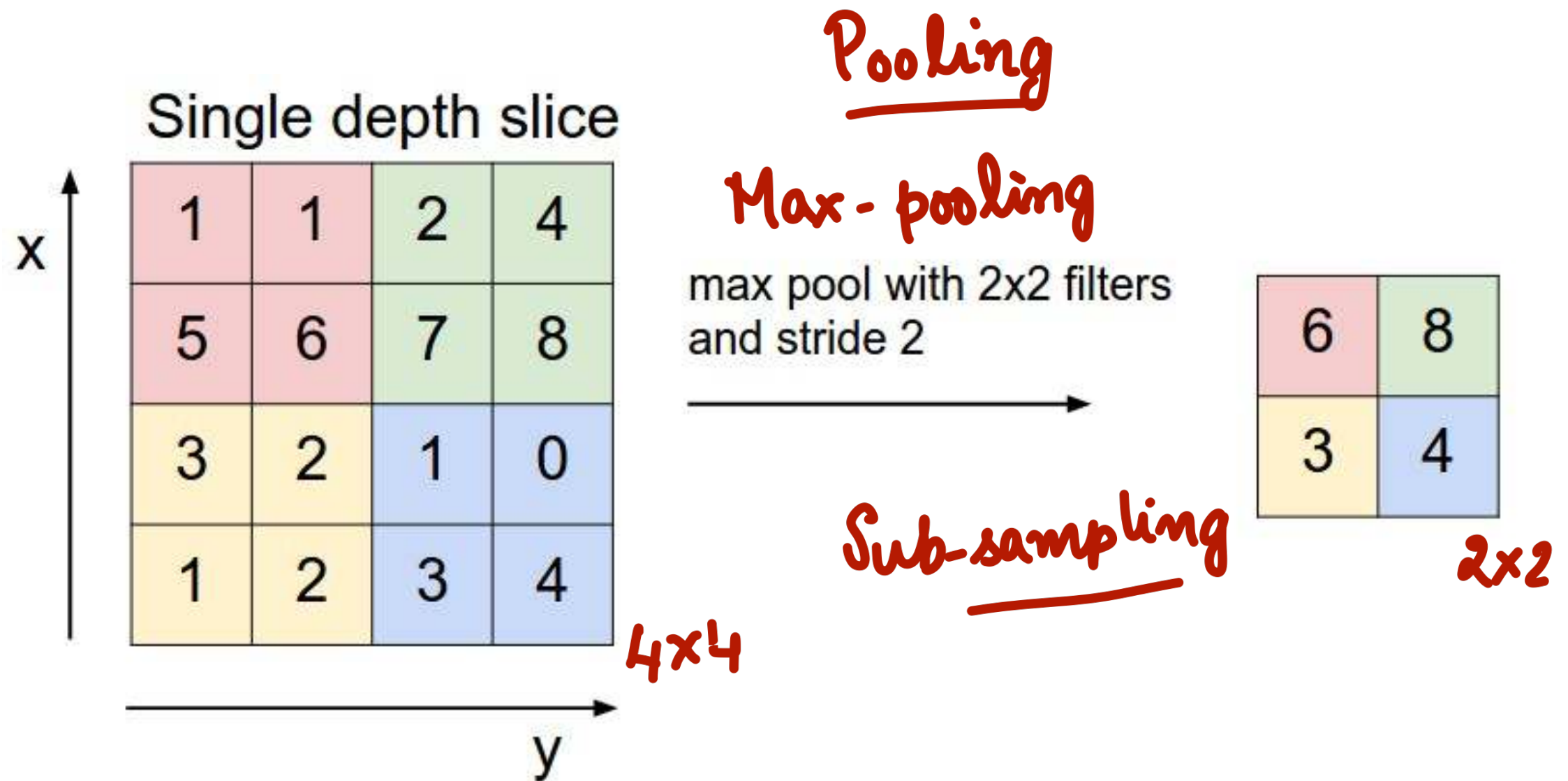
$$(Wx + b)$$

1-D

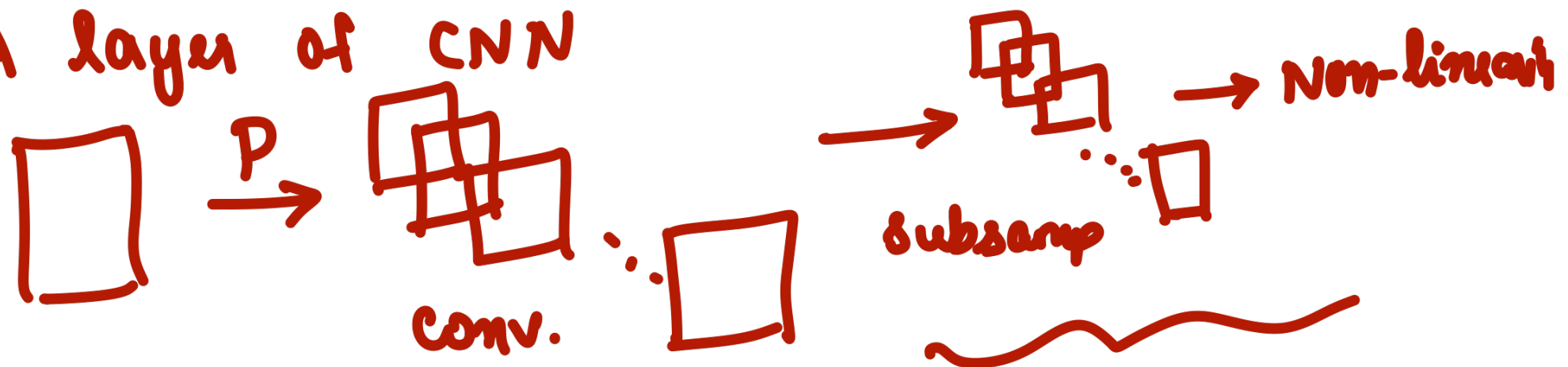
Destroys 2-D structure



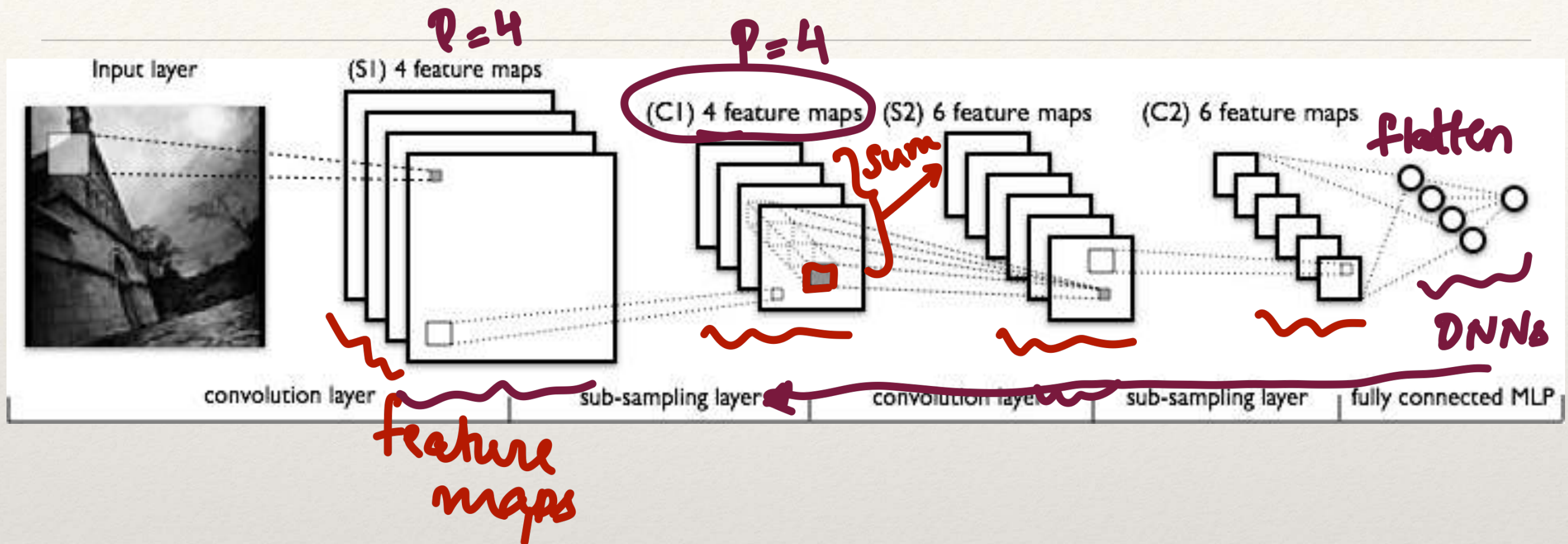
Max Pooling Operation



A layer of CNN

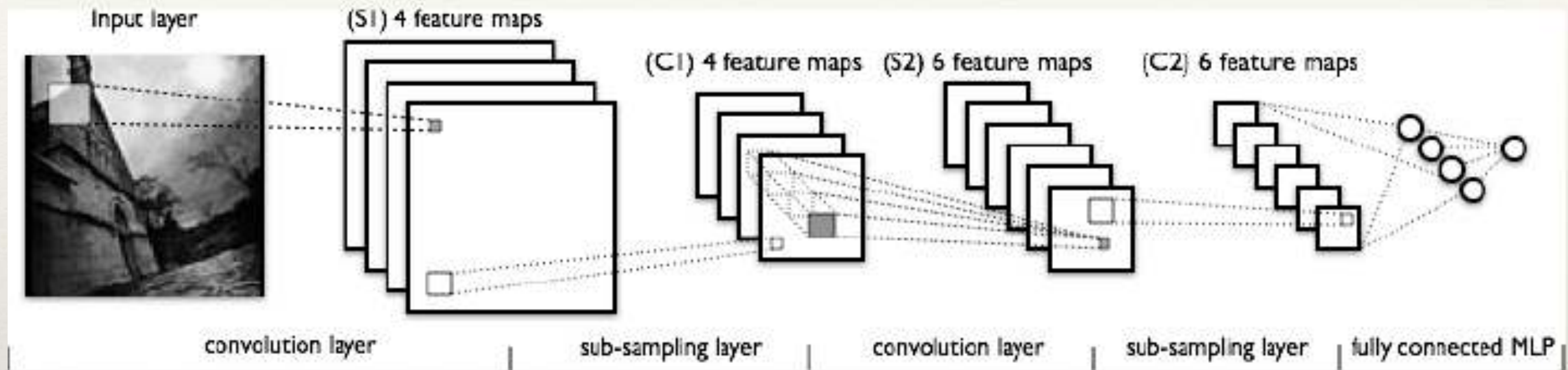


Convolutional Neural Networks



- Multiple levels of filtering and subsampling operations.
- Feature maps are generated at every layer.

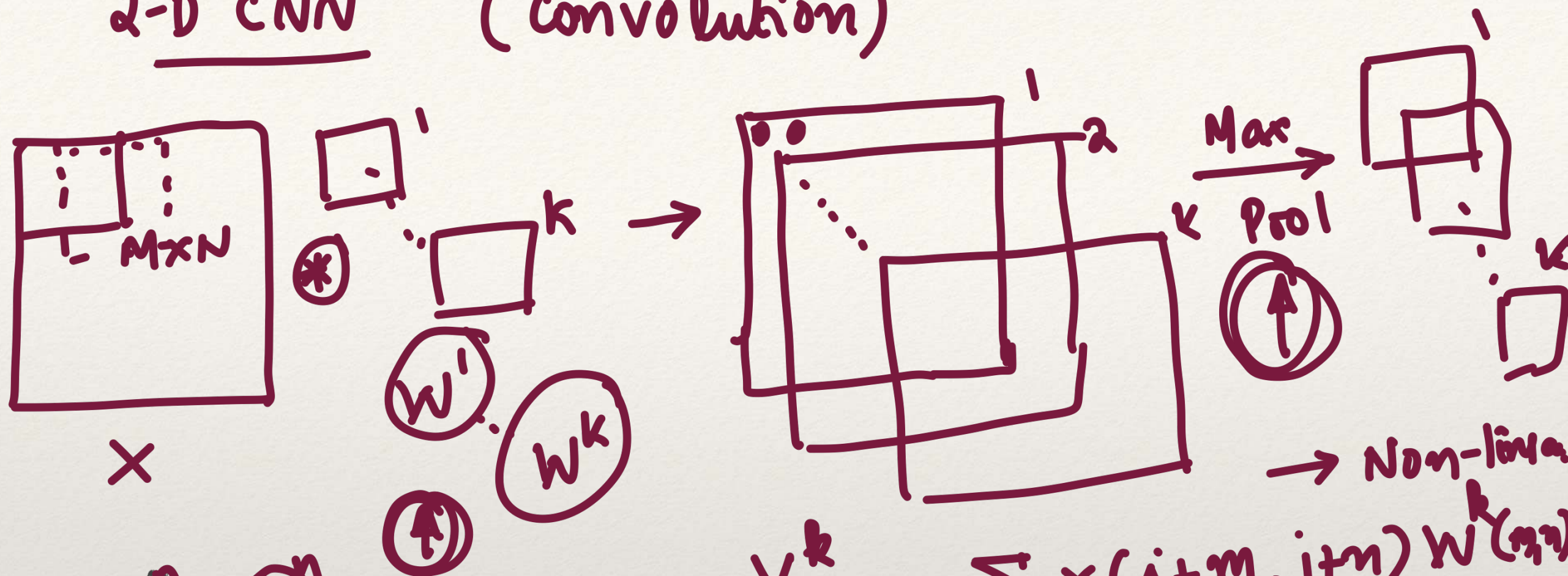
Convolutional Neural Networks



- Multiple levels of filtering and subsampling operations.
- Feature maps are generated at every layer.

Back Propagation in CNNs

2-D CNN (Convolution)



$$E = \sum_n E[t^n, y^n]$$

$$\frac{\partial E}{\partial W^k}$$

$$X \in \mathbb{R}^{D_1 \times D_2}$$

If no padding

$$\frac{\partial E}{\partial y^k}$$

$$\frac{\partial E}{\partial W^k}$$

$$y^k \in \mathbb{R}^{D_1 - M, D_2 - N}$$

$$y^k = \sum_{m,n} x(i+m, j+n) W^k(m,n)$$

Non-linear

$$\underline{\underline{y^k[0,0]}} = \underbrace{x[0,0]}_{\text{convolution}} \underbrace{w^k[0,0]}_{\text{scalar}} + \underline{x[0,1]} \underline{w^k[0,1]} + \dots + x[0,N] w^k[0,N] + x[1,0] w^k[1,0] + \dots + x[M,N] w^k[M,N]$$

$$\underline{\underline{y^k[0,1]}} = \underline{x[0,1]} \underbrace{w^k[0,0]}_{\text{scalar}} + \dots + x[0,N+1] w^k[0,N] + x[1,1] w^k[1,0] + \dots + x[M+1,N] w^k[M,N]$$

$$\vdots$$

$$\begin{bmatrix} \frac{\partial E}{\partial y^k} \end{bmatrix}_{(D_1-M, D_2-N)} \parallel$$

$$\rightarrow \begin{bmatrix} \frac{\partial E}{\partial w^k} \end{bmatrix}_{M+1, N+1} \parallel$$

$[w^k]_{M+1, N+1}$



$M+1, N+1$

$$\frac{\partial \mathcal{E}}{\partial w^k[0,0]} = x[0,0] \frac{\partial \mathcal{E}}{\partial y^k[0,0]} + \dots + x[D_1-M, D_2-N] \frac{\partial \mathcal{E}}{\partial y^k[D_1-M, D_2-N]} + x[0,1] \frac{\partial \mathcal{E}}{\partial y^k[0,1]}$$

$$(*) \quad \frac{\partial \mathcal{E}}{\partial w^k[0,1]} = \underline{x[0,1]} \frac{\partial \mathcal{E}}{\partial y^k[0,0]} + \dots + x[D_1-M, D_2-N+1] \frac{\partial \mathcal{E}}{\partial y^k[D_1-M, D_2-N+1]}$$

$$\vdots$$

$$\underline{N,N}$$

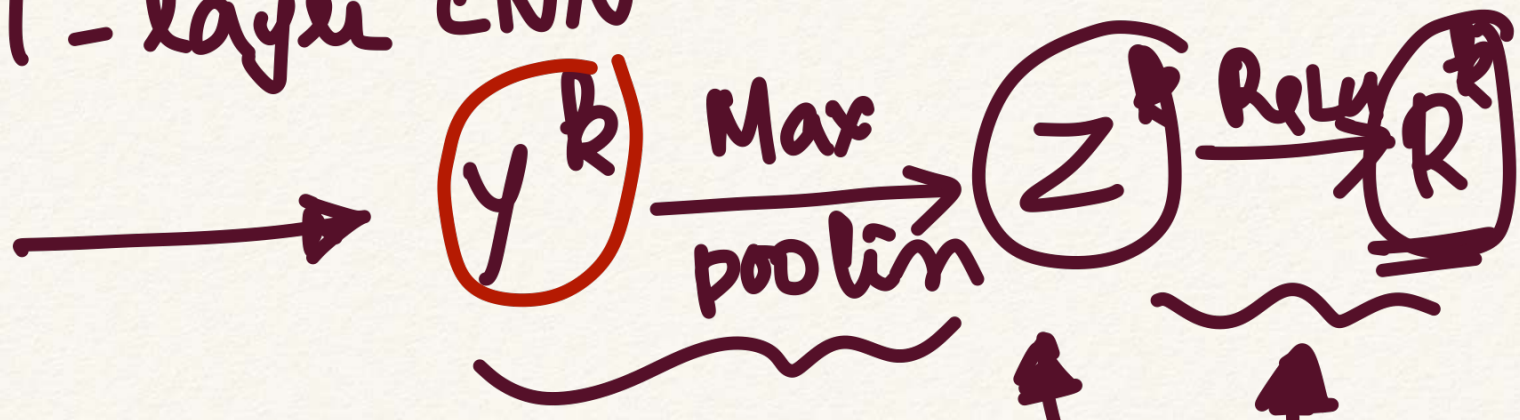
$$\frac{\partial \mathcal{E}}{\partial y^k[D_1-M, D_2-N]}$$

$$\frac{\partial \mathcal{E}}{\partial w^k[m,n]} = \sum_{\substack{p,q=0,0 \\ \text{conv.}}}^{D_1-M, D_2-N} x[m+p, n+q] \frac{\partial \mathcal{E}}{\partial y^k[p,q]}$$

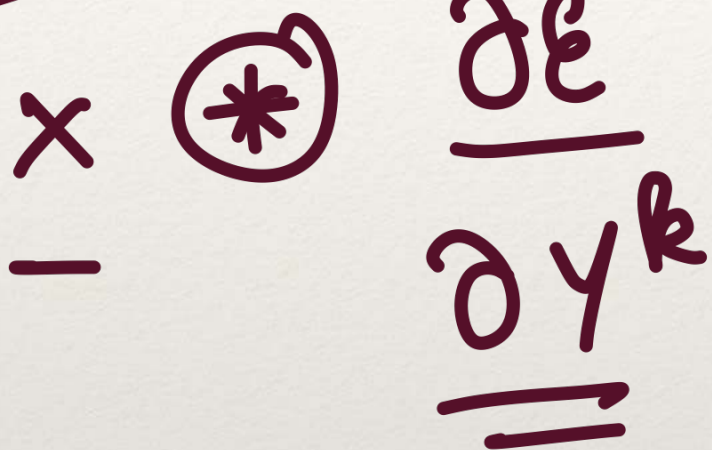
Forward



1-layer CNN

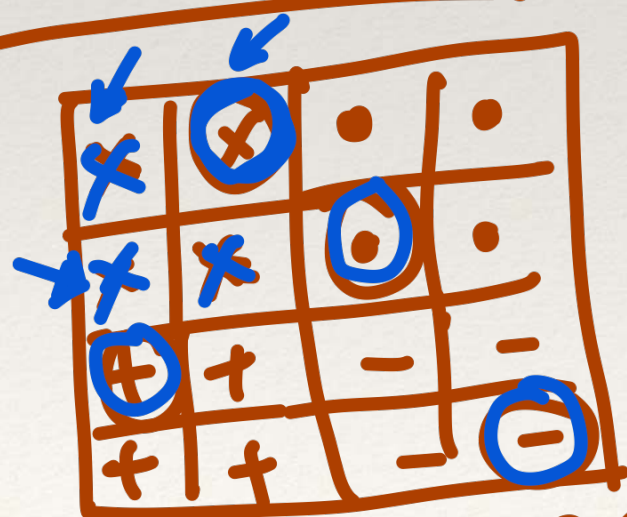


Backward

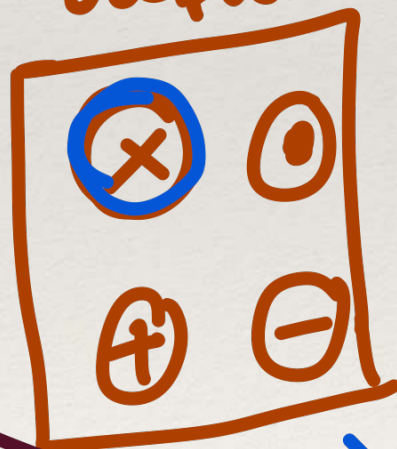


$$\frac{\partial}{\partial L} (f(L)) = \frac{\partial f}{\partial L} \cdot \frac{\partial L}{\partial L} = \frac{\partial f}{\partial L}$$

Max Pooling

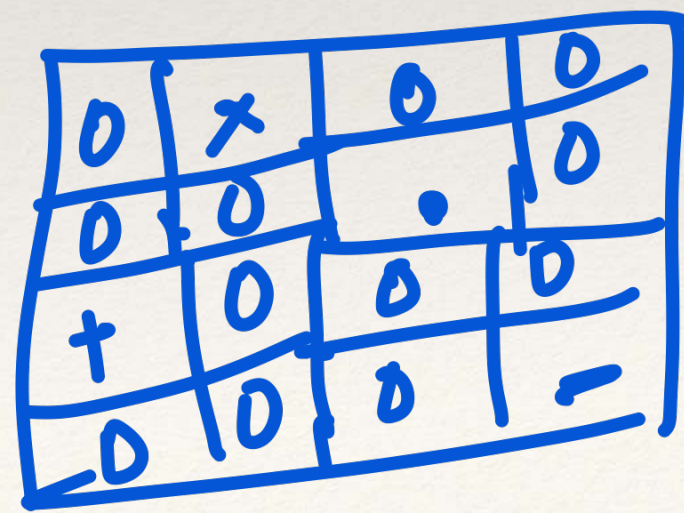


output



$$\frac{\partial \mathcal{E}}{\partial Z^k}$$

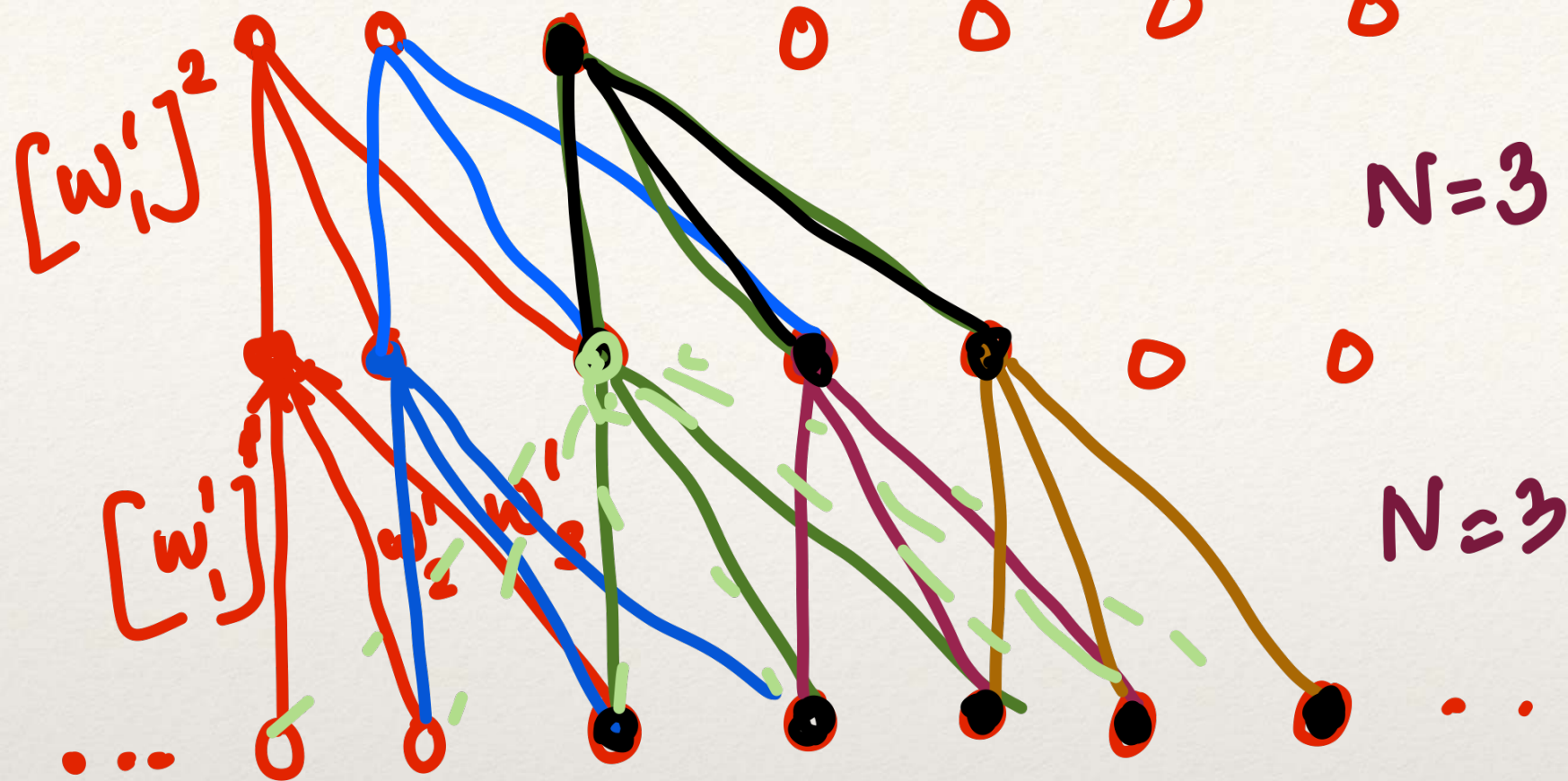
Unpooling



$$\frac{\partial \mathcal{E}}{\partial Z^k}$$

argmax

1-D Convolutional Network



2nd layer

1 kernel output
 k -such outputs k .
 w_k $i \rightarrow$ weight
 $x[n]$

Layer in more depth are looking at more and more global properties of the input

for i^{th} lag and k^{th} kernel

$$\underline{w}^2 = \begin{bmatrix} 0 \\ 1 \\ 3 \end{bmatrix} \quad \underline{w}' = \begin{bmatrix} 1/2 \\ -1 \\ 2 \end{bmatrix} \dots$$

Expression for a 1-D CNN

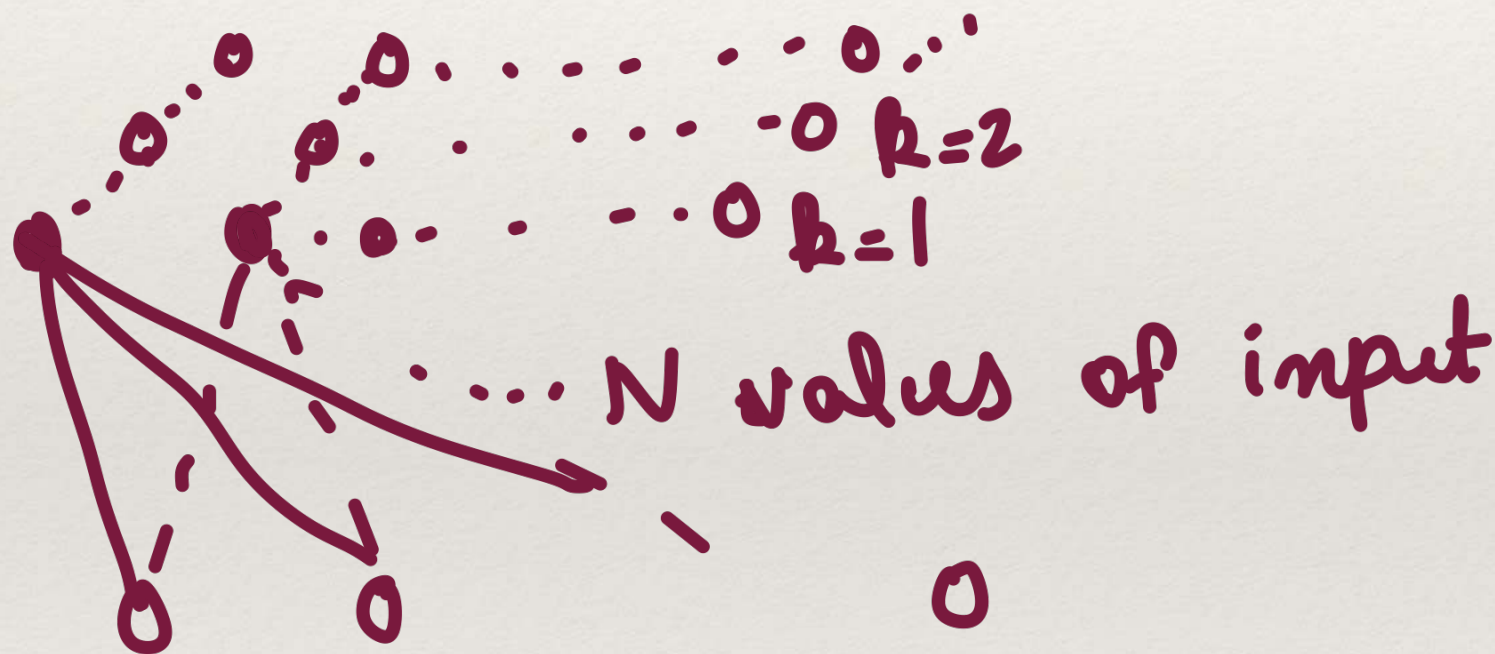
$$\underline{y^k[n]} = \sum_{i=1}^N \underline{x[n+i]} \underline{w_{i,k}^k}$$

N - kernel width

K - # kernels

$$k = \underline{1 \dots K}$$

$k-k$



parameters is $\underline{k \times N}$

$$\underline{w^0} = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \underline{w^2} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$